AIMERZ.ai
Aim.Act.Achieve

NOTES

Strings in Java

Working with String Object

Working with String Object

Understanding Strings

A string in Java is a sequence of characters. It's one of the most fundamental data types used to represent text. Think of it as a necklace where each bead is a character, and the necklace itself is the string.

Key Characteristics:

- ➔ **Immutability:** Once a string is created, its value cannot be changed. If you need to modify it, a new string is created.
- ➔ **Methods:** Java provides numerous methods to manipulate strings, such as length(), charAt(), substring(), toUpperCase(), and more.

Creating Strings

We can create strings in two primary ways:

1. **String Literals:**

```java
String greeting = "Hello, World!";
```

2. **Using the new Keyword:**

```java
String name = new String("Alice");
```

In Java, **String** is a class that represents sequences of characters. And strings are widely used in Java programming, and Java provides several features and methods to work with strings efficiently.

What is Immutable means:

Once a String literal object is created, its value cannot be changed. Any modification to a string results in the creation of a new String object.

Declaring a String Literal

Example:

```
Java
String s1 = "Hello";
s1.concat(" World");  // This doesn't modify s1
System.out.println(s1);  // Output: Hello
s1 = s1.concat(" World");  // Now s1 refers to the new "Hello World"
object
System.out.println(s1);  // Output: Hello World
```

In the above example code we can see clearly that **s1** which is holding "Hello", we cannot add another word to it as we are trying to add "World" to it by using concat() method, meaning concatenation is happening but String is immutable that is why the value of **s1** is not updating as HelloWorld.

Basically this concat() method merges two String values

But when we merge by writing s1 = s1.concat("World"); and If we see this, it is like s1 = "Hello" + "World"

In above code **s1** is holding a new string now which is "Hello World"

String Pool / String Constant Pool:

In Java, memory management for strings is optimized using something called the **String Pool** or **String Constant Pool**. The **string pool** is a special memory region inside the heap, managed by the JVM specifically for storing string literals and strings created using the `intern()` method. Here's how it works:

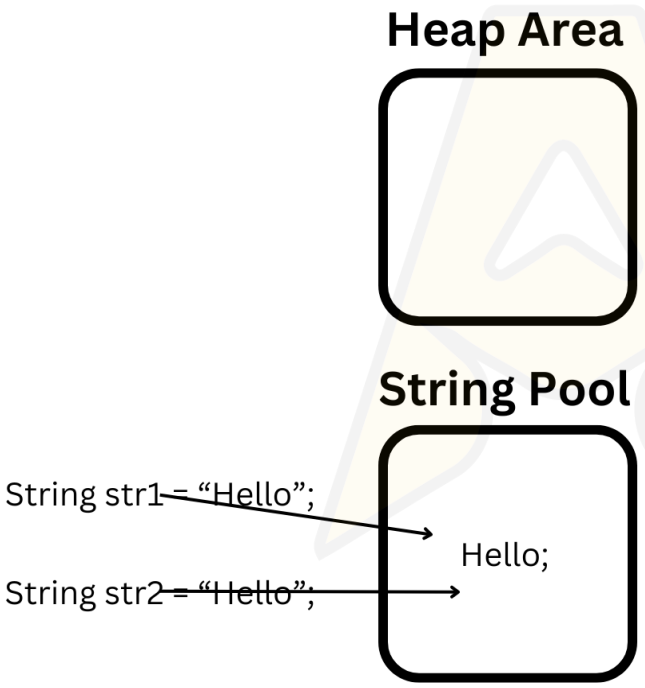
- When you create a **string using double quotes** (like `String s1 = "Hello";`), Java stores that string in a special area of memory called the **String Pool**.
- If another string is created with the **same content** (like `String s2 = "Hello";`), Java **won't create a new string**. Instead, it will **point** to the existing string in the pool.

Meaning in String Constant Pool **duplicates are not allowed**

Example:

```java
String str1 = "Hello";
String str2 = "Hello";  // str2 points to the same object as str1
System.out.println(s1 == s2);  // Output: true (both refer to the
same object)
```

Let's see above code in diagrammatic manner



In the above example code when we declare a string **str1** and define it with "**Hello**", then a string object is created in String Pool.
Again we declared another string **str2** and defined it with "**Hello**", but this time no other string object was created in String Pool. Because there is already a String Object with name "**Hello**" which has been created by **str1,** so **str2** object will pointing to the **str1** object because **str1** is also holding same object

And finally we are checking if both **str1** and **str2** are pointing to same object in string pool by this **== (equals)** operator, and we are able to see the output as **true**

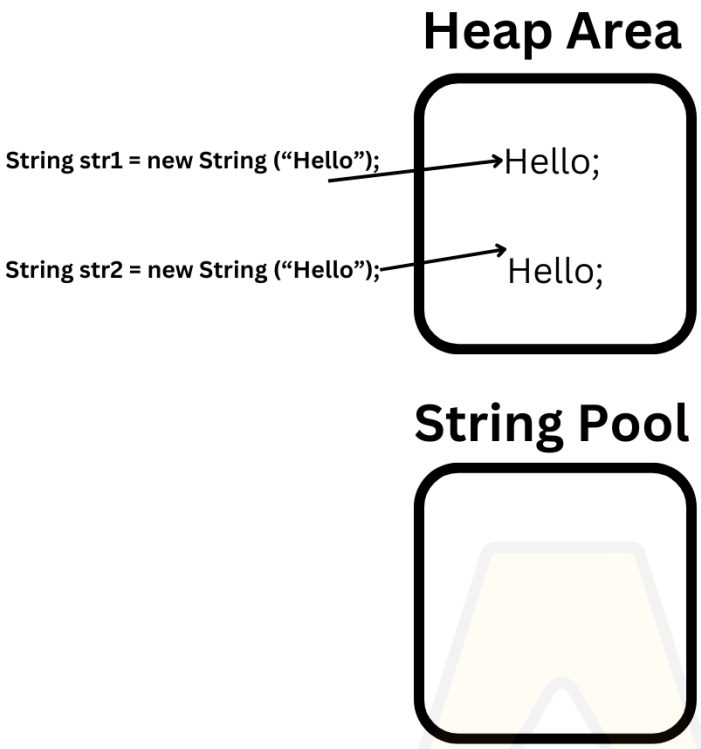
Using new keyword to create Strings:
We can create a new String object using the new keyword, which bypasses the string pool and creates a new object in the heap area

Declaring a String using new keyword

Example

```java
String str1 = new String("Hello");
String str2 = new String("Hello");
System.out.println(s1 == s2);  // Output: false (different objects in heap memory)
```

Let's understand this in a diagrammatic way



As we can see that when we are declaring String using a new keyword by calling the constructor of the String class which is accepting a string type value.
After defining **str1** with "**Hello**" we can see "**Hello**" is stored in Heap Area not in String Pool and for **str2** also got stored in Heap Area and two different objects created which are holding the same type of data.

That is why when we check references with **==** operator we are getting false. Meaning in Heap Area objects are newly created

Important Methods in the String Class

Method	Description
length()	Returns the length of the string (number of characters).
charAt(index)	Returns the character at the specified index.
substring(beginIndex)	Returns a new string starting from beginIndex to the end of the original string.

substring(beginIndex, endIndex)	Returns a substring from beginIndex to endIndex (excluding endIndex).
indexOf(char)	Returns the index of the first occurrence of the specified character.
lastIndexOf(char)	Returns the index of the last occurrence of the specified character.
contains(CharSequence)	Checks if the string contains the specified sequence of characters.
equals(Object)	Compares two strings for equality. Returns true if the strings are equal.
equalsIgnoreCase(String)	Compares two strings, ignoring case considerations.
toUpperCase()	Converts all characters in the string to uppercase.
toLowerCase()	Converts all characters in the string to lowercase.
replace(oldChar, newChar)	Replaces all occurrences of oldChar with newChar in the string.
trim()	Removes leading and trailing white spaces.
split(String regex)	Splits the string around matches of the given regular expression.

Examples using String class Methods

Java
```java
public class Main {
    public static void main(String[] args) {
        String str = "  Java Programming  ";

   System.out.println(str.length());          // Output: 20
   System.out.println(str.charAt(7));         // Output: P
   System.out.println(str.substring(2, 7));   // Output: Java
   System.out.println(str.indexOf('a'));      // Output: 3
   System.out.println(str.trim());            // Output: "Java
Programming" (removes spaces)
   System.out.println(str.toUpperCase());     // Output: "  JAVA
PROGRAMMING  "
```

```
    System.out.println(str.replace("Java", "C++"));  // Output: "  C++
Programming  "
    }
}
```

String Comparison in Java

equals(): This method compares the actual content of two strings.

Example:

```Java
String s1 = "Hello";
String s2 = "Hello";
System.out.println(s1.equals(s2));  // Output: true
```

== Operator:

Compares object references, not content. It checks if two strings refer to the same object.

Example:

```Java
String s1 = new String("Hello");
String s2 = new String("Hello");
System.out.println(s1 == s2);  // Output: false
(different objects in memory)
```

compareTo():

Compare two strings lexicographically. It returns:
0 if the strings are equal, or
a negative number if the first string comes before the second, or
a positive number if the first string comes after the second.

Example:

```Java
String s1 = "Apple";
String s2 = "Banana";
System.out.println(s1.compareTo(s2));  // Output: -1
(because "Apple" comes before "Banana")
```

StringBuilder and StringBuffer

In Java, **strings are immutable**, which means once we create a string, we cannot change it. If we try to change it, a new string is created. This can cause performance issues if you're doing a lot of string changes (like adding text together).

To solve this problem, Java provides **StringBuilder** and **StringBuffer**. These allow you to change (mutate) strings without creating new objects every time.

Key Points:

1. **StringBuilder** and **StringBuffer** help you **build strings** by allowing changes without making new objects each time.
2. The difference is mainly about **speed** and **thread safety**.

StringBuilder (Faster but Not Thread-Safe)

- ➔ **StringBuilder** is faster because it doesn't worry about **multiple threads** changing the string at the same time.
- ➔ If you **don't need to handle multiple threads** (for example, if you're only working in a single thread), **StringBuilder** is the better choice.
- ➔ Since it is not synchronized, it focuses on speed but can't protect data if multiple threads are involved.

Example of StringBuilder:

```Java
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World!");  // You can change the string without
creating a new object
```

```
System.out.println(sb); // This will print: Hello World!
```

StringBuffer (Thread-Safe but Slower)

- **StringBuffer** is **slower** than StringBuilder because it is **synchronized**, meaning it is **safe to use with multiple threads** at the same time.
- You should use **StringBuffer** when you need to ensure that multiple threads **don't change the string at the same time** and mess up the result.

Example of StringBuffer:

```Java
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World!");  // Similar to StringBuilder but thread-safe
System.out.println(sb); // This will also print: Hello World!
```

Difference Between StringBuilder and StringBuffer

Feature	StringBuilder	StringBuffer
Mutability	Mutable (can be changed)	Mutable (can be changed)
Thread Safety	**Not thread-safe** (not synchronized)	**Thread-safe** (synchronized)
Performance	**Faster** because no synchronization	**Slower** because of synchronization
Use Case	Use when working in a **single-threaded** environment	Use when working in a **multi-threaded** environment
Availability	Available since Java 5	Available since the beginning (Java 1)

Best For	Best when you need **high performance** and don't care about thread safety	Best when you need **thread safety**
Example	StringBuilder sb = new StringBuilder("Hi");	StringBuffer sb = new StringBuffer("Hi");

Methods in StringBuilder and StringBuffer:

append(String str): Appends the specified string to the sequence.

insert(int offset, String str): Inserts the string at the specified position.

delete(int start, int end): Deletes a substring from the sequence.

reverse(): Reverses the character sequence.

Example:

```Java
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
sb.insert(5, "is Awesome ");
System.out.println(sb.toString());// Output: Java is Awesome Programming
sb.reverse();
System.out.println(sb.toString());// Output: gnimmargorP emosewA si avaJ
```

String Manipulation

String manipulation in Java refers to working with strings, like changing, combining, or analyzing them. You can use built-in methods like length() to find the size, concat() to join strings, substring() to get a part of a string, and replace() to swap characters. Java strings are immutable, meaning once created, they can't be changed, but you can create new strings with modifications.

Example

```Java
public class Main {
```

```
    public static void main(String[] args) {
    String sentence = "Welcome to Aimerz Lets get a Job";

    // 1. Find the length of the string
    System.out.println("Length: " + sentence.length());

    // 2. Extract a substring
    String substring = sentence.substring(0, 17);
    System.out.println("Substring: " + substring);// Output: Welcome
to Aimerz
    // 3. Replace a word
     String replaced = sentence.replace("Lets", "I");
     System.out.println("Replaced: " + replaced);
// Output: Welcome to Aimerz I get a Job

        // 4. Convert to uppercase
  System.out.println("Uppercase: " + sentence.toUpperCase());

        // 5. Split the string into words
      String[] words = sentence.split(" ");
      for (String word : words) {
            System.out.println(word);
      }

        // 6. Check if the string contains a word
System.out.println("Contains 'fox': " + sentence.contains("Aimerz"));
    }
}
```

Output:

Length: 32
Substring: Welcome to Aimerz
Replaced: Welcome to Aimerz I get a Job
Uppercase: WELCOME TO AIMERZ I GET A JOB
Contains ‘Aimerz’: true

String Immutability and Memory Efficiency Example

String immutability makes string manipulation safer, but can also lead to inefficiencies in memory. For instance, repeatedly modifying strings in a loop can create many unnecessary objects.

Inefficient String Concatenation:

Example

```java
public class Main {
    public static void main(String[] args) {
        String result = "";
        for (int i = 0; i < 100; i++) {
            result += i;// Creates new String object each time
        }
        System.out.println(result);
    }
}
```

Efficient String Concatenation with StringBuilder:

Example

```java
public class Main {
    public static void main(String[] args) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < 100; i++) {
            result.append(i);// Efficient, uses a single object
        }
        System.out.println(result.toString());
    }
}
```

Real-World Examples

- **Text Processing:** Parsing user input, formatting text, and extracting information from strings.
- **Web Development:** Handling URL parameters, rendering HTML content, and working with JSON data.
- **Data Analysis:** Processing text data for tasks like sentiment analysis, keyword extraction, and natural language processing.

String vs. StringBuilder

Aspect	String	StringBuilder
Mutability	Immutable (cannot change once created)	Mutable (can modify without new Objects)
Efficiency	Slower for repeated modifications	Faster for repeated modifications
ThreadSafety	Thread-safe (safe in multithreaded apps)	Not thread-safe (use in single threads)
Use Case	When the value should remain constant	When string modifications are frequent

Inshort we can say **StringBuilder** is a more efficient choice than **String** when we need to frequently modify strings, such as adding, removing, or inserting characters. This is because **StringBuilder** is designed for efficient string manipulation, avoiding the creation of new objects with each change.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan